

Pipeline de Dados End-to-End para Plataforma de Streaming

Arquitetura Medalhão • Lakehouse Self-Hosted • Streaming Analytics

PostgreSQL

MinIO

Apache Airflow

PySpark

Delta Lake

Metabase

Docker



O Domínio

Plataforma de streaming estilo Twitch: streamers transmitem ao vivo, viewers assistem, assinam e doam. Eventos acontecem em alta frequência e volume.



O Problema

Dados espalhados em 13 tabelas relacionais. Sem análise consolidada: impossível responder 'qual streamer gera mais receita?' ou 'qual jogo atrai mais viewers?'



O que a Pipeline Resolve

Extrai, transforma e modela os dados em um Star Schema (Kimball) pronto para análise — com KPIs de receita, transmissões, viewers e popularidade de jogos, atualizados via DAG orquestrada pelo Airflow.

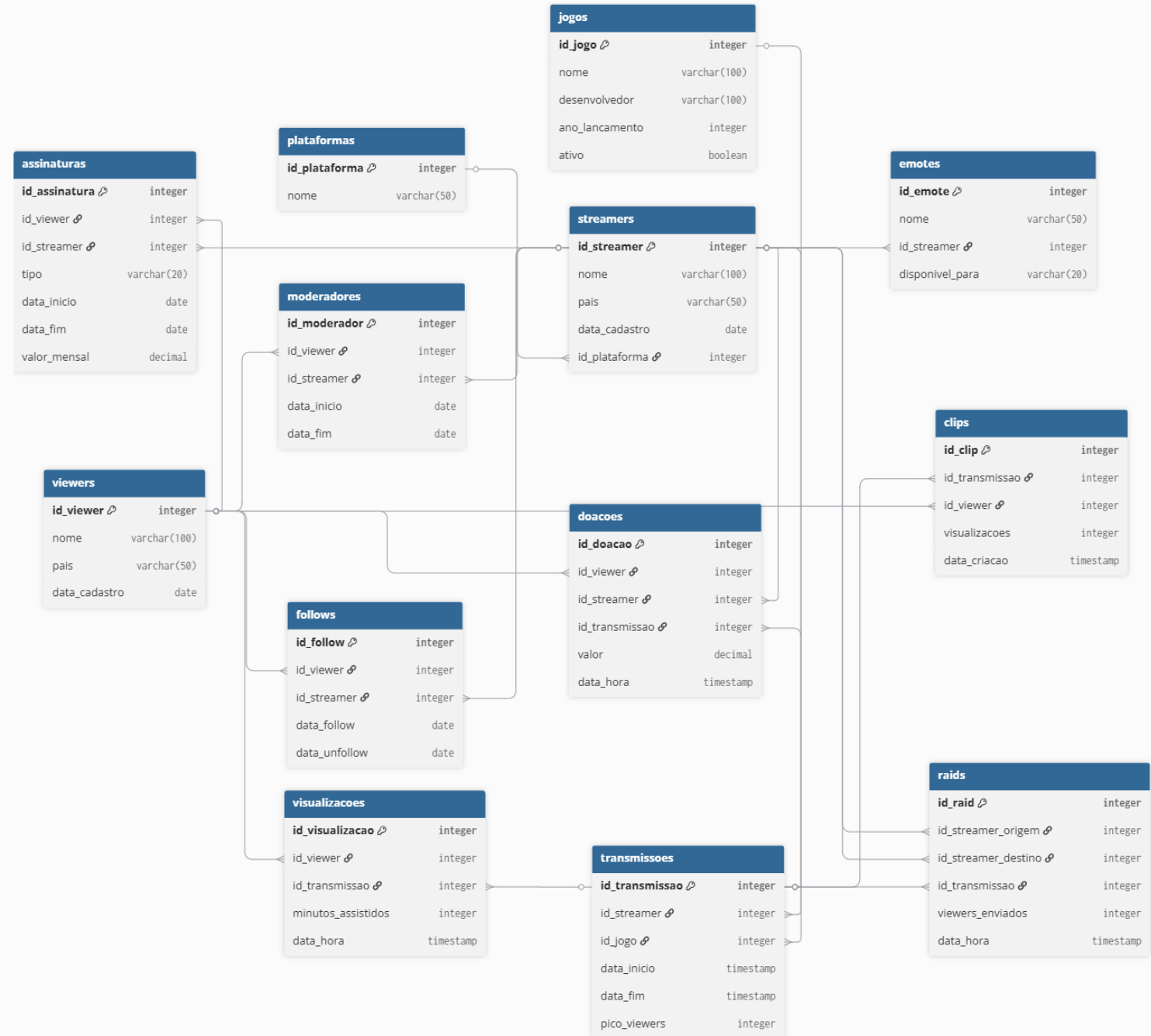


~110 mil registros sintéticos gerados com Faker representando o ciclo completo de uma plataforma de streaming.

Ambiente Relacional — Modelo ER

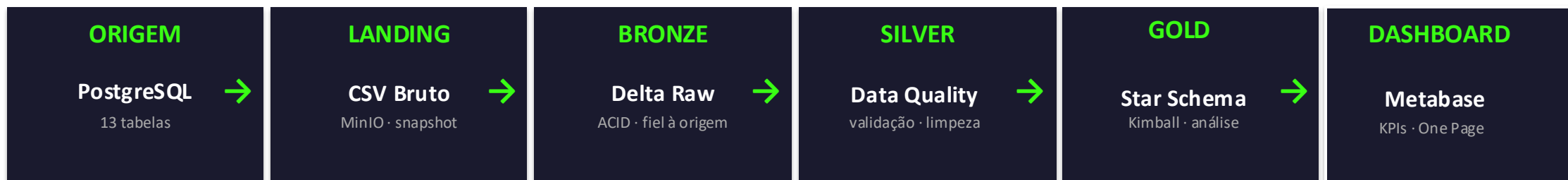
Banco de Origem · 13 Tabelas

PostgreSQL 15 via Docker · schema.sql inicializa automaticamente ·
Supabase usado como banco de origem compartilhado



Arquitetura Medalhão

Landing → Bronze → Silver → Gold



Data Lake — MinIO

Armazena todas as camadas (landing/bronze/silver/gold) como buckets S3-compatíveis. Delta Lake garante transações ACID e Time Travel.



Orquestração — Airflow

DAG pipeline_medallion encadeia todas as etapas em sequência. LocalExecutor, sem cron do SO, acionamento manual ou agendado.

Todos os serviços na mesma rede Docker datalake — comunicação por nome de container, sem configuração extra.

 not_null

Colunas obrigatórias validadas por tabela.
Registros com valores nulos são rejeitados e enviados à quarentena.

 domains

Valores aceitos por coluna definidos por tabela (ex: tier de assinatura aceita apenas 1, 2 ou 3).

 temporal

Datas coerentes: data_fim >= data_inicio,
timestamps dentro de intervalo válido.

 Quarentena

Registros inválidos gravados em:
s3a://silver/_quarentena/<tabela>/

Rastreáveis para auditoria posterior.

 Silver Válida

Dados aprovados gravados em:
s3a://silver/<tabela>/

Delta Lake · ACID · Time Travel

 Separação entre dado válido e quarentena garante rastreabilidade e confiabilidade para as camadas Gold e Dashboard.

6 Ambiente Dimensional — Star Schema (Kimball)

Gold Layer · 5 Dimensões · 4 Fatos

 **dim_tempo**

 **dim_jogo**

 **dim_plataforma**

 **dim_streamer**

 **dim_viewer**

 **FATOS**

transmissoes
visualizacoes
doacoes · assinaturas

Surrogate Keys

Cada dimensão tem sk_* gerado artificialmente. Membro desconhecido sk=-1 garante integridade referencial entre fato e dimensão.

Data Marts (Gold)

agg_streamer_visao_geral
agg_receita_mensal
agg_jogo_popularidade
agg_plataforma_resumo

 **Marts exportados via JDBC para schema gold_analytics no PostgreSQL → consumo pelo Metabase**

Infraestrutura — 100% Self-Hosted

Docker · Rede datalake compartilhada

Servico	Tecnologia	Porta	Papel
postgres_origem	PostgreSQL 15	5433	Banco de origem — inicializa schema.sql automaticamente
minio	MinIO	9000 / 9001	Data Lake S3-compatível — buckets landing/bronze/silver/gold
jupyter	Jupyter + Spark	8888	Exploração, desenvolvimento e testes dos scripts
airflow	Apache Airflow 2.9	8080	Orquestração da DAG pipeline_medallion (LocalExecutor)
metabase	Metabase	3000	Dashboard One Page View — lê schema gold_analytics



Todos os serviços na mesma rede Docker datalake — comunicação por nome de container, sem IP fixo ou configuração extra.



Detalhes Técnicos

- Retries: 1 · delay: 2 min
- Schedule: manual (sem cron do SO)
- POSTGRES_HOST sobrescrito por container
 - metabase_setup é idempotente



Como Acessar

URL: <http://localhost:8080>

DAG: pipeline_medallion

Trigger manual → acompanha execução em tempo real no grafo da DAG



O Airflow é o único ponto de controle — nenhuma etapa precisa ser disparada manualmente fora dele.

9 Dashboard — Metabase One Page View

KPIs · Métricas · Self-hosted

 KPI 1

Receita Total

SUM(receita_total)
agg_receita_mensal

 KPI 2

Ticket Médio por Doação

receita_doacoes / qtd_doacoes
agg_receita_mensal

 KPI 3

Total de Transmissões

SUM(qtd_transmissoes)
agg_streamer_visao_geral

 KPI 4

Viewers Ativos

SUM(viewers_unicos)
agg_streamer_visao_geral

Gold (Delta/MinIO) → gold_to_postgres (JDBC overwrite) → schema gold_analytics → Metabase lê via SQL padrão

10 Demonstração ao Vivo + Diferenciais

Roteiro · GitHub · MkDocs

1 **Mostrar GitHub + MkDocs**
Repositório, README e documentação publicada em GitHub Pages

2 **Modelo ER no banco**
schema.sql / Supabase com 13 tabelas populadas

3 **Disparar a DAG no Airflow**
localhost:8080 · pipeline_medallion · Trigger manual

4 **Dados no MinIO**
Buckets landing/bronze/silver/gold populados após execução

5 **Dashboard Metabase**
KPIs atualizados em localhost:3000 após a DAG

★ Diferenciais do Projeto

- 100% self-hosted via Docker
- Delta Lake com ACID + Time Travel
- Data Quality com quarentena auditável
 - DAG idempotente e rastreável
- Documentação publicada (MkDocs Material)

👥 Equipe

Davi Novakoski · Victor Casagrande
Isabela Madeira José · Isaac Alexsander
Vitor Baschiroto

[github.com/davinovakoskim-code/
projeto-final-eng-dados](https://github.com/davinovakoskim-code/projeto-final-eng-dados)